

Trabalho Final — 13 Temas Disponíveis

Testes de Aplicações Mobile · PUC Minas IEC · Prof. Jackson Smith Moisés Matias

Cada grupo (3–4 alunos) escolhe **um tema** e desenvolve um artefato técnico completo cobrindo esse tema. Organizados por **trilha de skill** — escolha pelo que seu grupo curte, não só pelo que parece “mais IA”. Cada tema tem uma pasta própria em `temas/NN-slug/README.md` no repo, com passo a passo de execução — esta tabela é o resumo.

Enunciado completo, rubrica e regras: [enunciado.md](#)



Automação & Arquitetura de Teste

#	Tema	O que é	Stack sugerida
1	Automação Mobile End-to-End ★★★★★	Testar o app inteiro como um usuário testaria — abrir, navegar, tocar em botão — automatizado, em vários celulares/emuladores.	Maestro. Cloud Devices (Firebase Test Lab/BrowserStack) é diferencial, não obrigação
2	Arquitetura de Suíte de Teste ★★	Organizar os testes de um jeito mais limpo e reutilizável — separar “o que testar” de “como testar”.	Robot Pattern sobre RNTL e/ou <code>runFlow</code> do Maestro em camadas reutilizáveis
5	Visual Regression Testing Mobile ★★	Testar se a tela está visualmente certa — comparar screenshot automaticamente.	Maestro Visual Testing (nativo) ou React Native Owl (open source)
9	Mobile API Contract Testing ★★★★★	Garantir que app e backend continuam “se entendendo” — se a API mudar sem avisar, o teste acusa.	Pact + Jest (consumer-driven contracts) — precisa ativar modo real com token TMDB

#	Tema	O que é	Stack sugerida
13	POM (Page Object Model) em Maestro ★★★★★	Separar o <code>id</code> de cada elemento de tela de como o teste interage com ele — renomear 1 testID conserta em 1 lugar, não em 5.	<code>runScript</code> + <code>output</code> do próprio Maestro — receita oficial da documentação

🧠 IA aplicada a testes mobile

#	Tema	O que é	Stack sugerida
6	Test Generation com LLM ★★★★★	Usar IA pra escrever testes automaticamente a partir de descrição em português do que o app deve fazer.	Claude API + Maestro (ou Claude Code via MCP)
7	AI Agent para Exploratory Mobile ★★★★★	Soltar um agente de IA pra explorar o app sozinho e reportar bugs por conta própria.	Claude Code + Maestro MCP (agentic exploration)

⚡ Performance & Segurança

#	Tema	O que é	Stack sugerida
3	Performance Mobile Testing ★★★★★	Medir se o app é rápido de verdade — cold start, jank, CPU/memória — com dado, não achismo.	Flashlight (CLI sobre ADB) + Reassure (Callstack)
4	Mobile Security Testing ★★★★★	Caçar falhas de segurança antes que um atacante ache — dado vazando, permissão desnecessária.	OWASP MASVS + achado manual de manifest (obrigatório) + MobSF (bônus)

🏗️ Qualidade & Pipeline

#	Tema	O que é	Stack sugerida
8	CI/CD Pipeline Mobile ★★★★	Montar a esteira automática que roda testes, builda e prepara o app pra publicar.	GitHub Actions + Fastlane (build+test lanes)
11	Mutation Testing ★★★	Testar os TESTES: muda um pedaço do código de propósito e vê se algum teste pega o erro.	Stryker (StrykerJS) sobre lib de domínio

Manual, Exploratório & Acessibilidade

#	Tema	O que é	Stack sugerida
10	Accessibility Testing Mobile ★★	Testar se o app funciona pra quem usa leitor de tela, tem baixa visão ou dificuldade motora.	Android Accessibility Scanner + auditoria manual WCAG
12	Testes Manuais Estruturados → Regressão Automatizada ★★★	Testar na unha com técnica publicada de verdade, depois converter cada bug achado em teste automatizado.	Parte A: heurística (TQED ou Gamified Exploratory Testing) · Parte B: conversão em RNTL/Maestro

Pré-requisito: device/emulador

9 dos 13 temas precisam de device/emulador Android em algum momento (temas 1, 3, 5, 6, 7, 10, 12 — precisam pra rodar de verdade; temas 4 e 8 só em parte).

Sem device confiável? 4 temas rodam 100% em código/API, sem abrir emulador: **tema 2** (Robot Pattern — Jest puro), **tema 9** (Contract Testing — Jest+Pact), **tema 11** (Mutation Testing — Stryker+Jest), **tema 13** (POM em Maestro — reorganiza seletores sem rodar de verdade, só valida no final).

Como compor a nota (60 pts)

Critério	Peso	Pontos
Apresentação oral estruturada (10min + 5min Q&A)	20%	12

Critério	Peso	Pontos
Artefato técnico (repo GitHub funcional, executável)	35%	21
Relatório analítico (1pg, conciso, foco em insights)	20%	12
Domínio conceitual demonstrado em Q&A	15%	9
Originalidade e profundidade	10%	6

Entrega: repo GitHub público + relatório 1 página + apresentação ao vivo (10min + 5min Q&A).

Dúvidas: Teams da turma ou jackson.96@gmail.com.